Vyapar Sathi Complete Revision Guide

How to Use This Guide

This document is a full replacement for the earlier conversation and scattered notes. Read this before interview, demo, viva, code explanation, or local setup. It is written as a revision guide, not as a short summary.

Core Identity

Vyapar Sathi is a full-stack AI-powered business assistant for small businesses. It combines transaction tracking, dashboard reporting, chat-based business Q and A, GST knowledge retrieval, bill OCR, and voice-to-text in one system.

1. One-Line Project Explanation

Vyapar Sathi is an AI-powered smart business diary that helps users manage transactions, scan bills, ask questions about business data, and get GST-related answers through both database querying and retrieval-based AI.

2. Elevator Pitch for Interview

Vyapar Sathi is a React plus FastAPI application built for Indian small businesses. It stores structured business data such as users, transactions, chat sessions, and uploaded document metadata in MySQL. It also uses ChromaDB as a vector database for GST knowledge retrieval. The system supports dashboard analytics, transaction CRUD, bill OCR, speech-to-text, and an AI assistant that decides whether a user query should be answered from MySQL, from GST documents, or from a combination of both.

3. Main Features of the Project

- User signup and login with JWT-based authentication
- Protected pages after successful login
- Dashboard with sales, expenses, profit, GST, charts, and recent transactions
- Transaction management with add, edit, delete, search, filter, and duplicate
- AI assistant for business data questions
- GST knowledge assistant using RAG with ChromaDB
- Bill upload and OCR extraction
- Voice upload and live browser microphone transcription
- Reports with charts and category analysis
- Persistent chat sessions and message history
- Multimodal workflow using text, image, and voice input

Feature	What It Does	Main Files
Authentication	Registers users, logs them in, returns JWT token	`app/api/auth_router.py`, `app/core/security.py`
Dashboard	Shows business KPIs and quick actions	`frontend/src/pages/dashboard-page.jsx`
Transactions	CRUD for income and expense records	`frontend/src/pages/transactions-page.jsx`, `app/api/transaction_router.py`
Assistant	Chat with business data and GST knowledge	`frontend/src/pages/assistant-page.jsx`, `app/api/chat_router.py`
OCR	Reads bills and extracts structured values	`app/services/ocr_service.py`, `app/api/multimodal_router.py`
STT	Converts uploaded or live-recorded audio to text	`app/services/stt_service.py`, `app/api/multimodal_router.py`
RAG	Retrieves GST knowledge from document chunks	`app/services/rag_service.py`
Reports	Builds charts and insight views from transaction data	`frontend/src/pages/reports-page.jsx`

| Feature | What It Does | Main Files |

4. Tech Stack Overview

Frontend

- React 18
- Vite
- React Router
- Tailwind CSS
- React Query
- Axios
- React Hook Form
- Zod
- Framer Motion
- Recharts
- Radix UI
- Sonner

Backend

- Python
- FastAPI
- Uvicorn
- SQLAlchemy
- Pydantic
- PyMySQL
- Passlib
- python-jose

AI and Multimodal

- ChromaDB
- Tesseract OCR with `pytesseract`
- Whisper-based speech-to-text
- LangChain loaders and splitters
- TinyLlama client
- PyPDF

Storage

- MySQL for structured application data
- ChromaDB for GST semantic retrieval
- Local filesystem `uploads/` for stored uploaded files

5. Active Application Structure

The active full-stack application uses:

- `frontend/` for the React frontend
- `app/` for the FastAPI backend
- `docs/` for interview and revision material
- `uploads/` for uploaded files

There are older prototype folders such as `step-1`, `step2-multimodal (1)`, and `step3-query-classifier`, but the main live system is the integrated React plus FastAPI app.

Important Clarification
If someone asks which code is the main project, answer: the current integrated project is in `frontend/` and `app/`. The `step-*` folders are previous development stages and experiments.

6. Frontend Architecture

The frontend is a single-page React application.

App startup

The frontend entry file `frontend/src/main.jsx` wraps the app with:

- `ThemeProvider`
- `QueryClientProvider`
- `AuthProvider`
- `BrowserRouter`
- `Toaster`

This means the app has:

- Theme state
- Centralized data fetching and caching
- Shared login state
- Client-side routing
- Toast notifications

Routing

Main routes defined in `frontend/src/App.jsx`:

- `/` -> auth page
- `/dashboard`
- `/transactions`
- `/assistant`
- `/upload`
- `/reports`

Protected pages are wrapped in `ProtectedRoute`, so users must be logged in to access them.

Frontend navigation flow

- Login or signup on auth page
- Redirect to dashboard after successful login
- Dashboard links to transactions, assistant, upload, and reports
- Bottom mobile navigation and desktop sidebar both support the same modules

7. Backend Architecture

The backend is layered cleanly:

- `app/main.py` -> FastAPI entry point
- `app/api/` -> route handlers
- `app/core/` -> database setup and security
- `app/models/` -> SQLAlchemy tables
- `app/schemas/` -> request and response schemas
- `app/services/` -> business logic and orchestration

This separation is one of the strongest architecture points in the project because it avoids mixing database logic, request validation, and business logic in the same file.

8. Database Architecture

The project uses two databases because it has two very different workloads.

Database	Purpose	Stores
MySQL	Structured operational data	Users, transactions, chat sessions, messages, uploaded file metadata
ChromaDB	Semantic retrieval for GST knowledge	Document chunks, embeddings, vector index information

Why MySQL

- Structured relational data
- Exact filtering and aggregation
- Good for total sales, expenses, profit, GST sums, date filters, and reporting

Why ChromaDB

- Supports meaning-based similarity search
- Better for GST knowledge retrieval than relational querying
- Stores vectorized document chunks persistently

Why not only MySQL

Because semantic retrieval over long GST text is not MySQL's main strength.

Why not only ChromaDB

Because ChromaDB is not meant to replace relational storage for users, transactions, and reports.

Why not MongoDB

Because this project is reporting-heavy and relational. Exact aggregation, structured data, and consistent finance-oriented records are a better fit for MySQL.

9. Main MySQL Tables You Should Remember

Table	Meaning
`users`	Stores account details
`user_sessions`	Stores refresh or session data
`user_profiles`	Stores business profile information
`conversations`	Stores chat session records
`messages`	Stores user and assistant chat messages
`transactions`	Stores business ledger entries
`user_documents`	Stores uploaded file metadata
`system_logs`	Stores logging and event-type data

Table	Meaning
`gst_documents`	Optional GST document storage metadata
`embeddings_metadata`	Embedding-related metadata

Primary key and foreign key examples

- `users.id` is the main user primary key
- `transactions.user_id` connects transactions to a user
- `messages.conversation_id` connects messages to a conversation
- `user_documents.conversation_id` can connect an uploaded file to a chat

10. Request and Response Schemas

The project uses Pydantic schemas in `app/schemas/schemas.py`.

Important schemas:

- `UserCreate`
- `UserResponse`
- `Token`
- `TransactionCreate`
- `TransactionUpdate`
- `TransactionResponse`
- `ChatRequest`
- `ChatResponse`
- `DashboardSummary`

Important concept:

- Models define database tables
- Schemas define API input and output

Good viva line

I used SQLAlchemy models for database structure and Pydantic schemas for request and response validation.

11. Authentication Flow

Register flow

1. Frontend sends `fullName`, `email`, and `password`
2. Backend validates input with `UserCreate`
3. Backend checks if email already exists
4. Password is hashed using Passlib Argon2
5. User is inserted into MySQL
6. API returns success response

Login flow

1. Frontend sends email as `username` plus password using form data
2. Backend searches the user by email
3. Backend verifies the password hash
4. If valid, JWT token is created
5. Frontend stores token in local storage
6. Future API requests attach the token in the `Authorization` header

Important files

- `app/api/auth_router.py`
- `app/core/security.py`
- `frontend/src/providers/auth-provider.jsx`
- `frontend/src/lib/api.js`

> **Authentication Insight**
> Signup stores the user in MySQL, but login succeeds only if the same email and password are sent again. A `401 Unauthorized` after a successful register call usually means credential mismatch or a backend environment mismatch, not that account creation failed.

12. User Flow Diagram Explanation

Full user journey in words

The user opens the app and first sees the auth page if not already logged in. After signup or login, the user reaches the dashboard. From the dashboard, the user can quickly add a transaction, navigate to the transaction workspace, open the AI assistant, upload a bill, or view reports. Transactions go to MySQL, assistant messages go through the orchestration layer, uploaded bills go through OCR, and reports are built from transaction data.

Interview-ready flow

`Login or Signup -> Dashboard -> Transactions / Assistant / Upload / Reports -> Save Data -> Show Insights`

Expanded user flow

1. User authenticates
2. Dashboard loads summary data
3. User can add, edit, or delete transactions
4. User can ask the assistant about sales, expenses, GST, or category totals
5. User can upload a bill and save OCR output as a transaction
6. User can use voice input for the assistant
7. User can review reports and trends

13. Data Flow Diagram Explanation

High-level data flow

- React frontend sends API requests
- FastAPI backend receives and validates them
- MySQL stores structured business data
- ChromaDB stores GST knowledge chunks and embeddings
- Filesystem stores uploaded files in `uploads/`
- OCR and STT services process uploaded or temporary files
- Chat orchestrator decides answer path and returns result

Interview-ready data flow

`Frontend -> FastAPI -> MySQL for business data`

`Frontend -> FastAPI -> ChromaDB for GST knowledge retrieval`

`Frontend -> FastAPI -> uploads folder -> OCR or STT -> assistant or transaction flow`

Data flow by module

Module	Input	Backend processing	Output
Auth	Email and password	Hashing, verification, JWT	User created or token returned
Transactions	Ledger fields	Validation and DB insert or update	Saved transaction
Assistant	User message	Classifier + DataService or RAGService	Answer
Upload	Bill image	Save file + OCR parse	Extracted fields
Voice	Audio blob or file	STT transcription	Text transcript
Reports	Transaction fetch	Aggregation and client-side charts	Insights and visuals

14. Dashboard Module

The dashboard is the user's overview screen.

It shows:

- Total sales
- Total expenses
- Net profit
- GST tracked
- Income vs expense chart
- Recent transactions
- Quick action buttons
- Lightweight AI insights built from live transaction data

APIs used:

- `GET /api/transactions`
- `GET /api/transaction-types`
- `GET /api/dashboard/summary`

15. Transactions Module

This is the structured ledger workspace.

User capabilities:

- Add transaction
- Edit transaction
- Delete transaction
- Duplicate transaction
- Search by category, note, or type
- Filter by transaction type

Transaction fields include:

- Date
- Type
- Amount
- GST amount
- Quantity
- Category
- Description

Important backend endpoints:

- `POST /api/transactions`
- `GET /api/transactions`
- `PUT /api/transactions/{id}`
- `DELETE /api/transactions/{id}`

16. Reports Module

The reports page gives visual business insights.

It includes:

- Total sales
- Total expenses
- Profit
- GST summary
- Profit trend line chart
- Top expense categories
- Income source breakdown
- Spending table

Important detail:

The reports page currently builds many insights client-side from fetched transaction data.

17. AI Assistant Module

The assistant is one of the strongest parts of the project.

User can:

- Ask typed business questions
- Ask GST questions
- Upload a bill
- Upload a voice note
- Use live microphone recording
- Open previous chat sessions
- Delete a chat

The frontend assistant page:

- Loads chat sessions
- Loads chat history
- Sends user message to backend
- Displays assistant answers
- Handles multimodal uploads

The backend assistant flow:

1. Accept text, image, or audio-related input
2. Convert image or voice to text if needed
3. Classify the query
4. Route query to MySQL logic or GST retrieval logic
5. Save messages to MySQL
6. Return final answer

18. Query Classification Logic

The classifier service decides whether a question is:

- `sql`
- `general`
- `mixed`

Examples

- "What is my total sales this month?" -> SQL
- "What is GST on mobile phones?" -> General
- "Tell me my rent and GST on rent" -> Mixed

Why this matters

The assistant does not answer every query using the same method. It first decides which source is most appropriate.

19. DataService Logic

`app/services/data_service.py` handles business-data answers from MySQL.

It can answer:

- Total sales
- Total expenses
- Profit
- GST total
- Average amount
- Highest or lowest transaction
- Recent transactions
- Transaction count
- Category totals
- Quantity totals
- Date-based filters such as today, this month, last month, and this year

This service is important because it turns natural language-like business questions into deterministic MySQL-based answers instead of relying purely on a generative model.

20. RAGService and GST Knowledge Retrieval

The GST assistant uses ChromaDB and document chunking.

Flow

1. Load GST PDFs
2. Split text into chunks
3. Convert chunks into embeddings
4. Store them in ChromaDB
5. Embed the user query
6. Retrieve nearest chunks
7. Build a concise grounded answer

Chunking values used in the code

- Chunk size: `1000`
- Chunk overlap: `150`

Meaning

- Chunk size controls how much context each chunk contains
- Overlap preserves context across chunk boundaries

Why not FAISS

FAISS is powerful but lower-level. ChromaDB is easier to use as a persistent application-oriented vector store for this project.

21. OCR Bill Processing

OCR is handled by `app/services/ocr_service.py`.

OCR flow

1. Accept uploaded image
2. Run Tesseract to extract raw text
3. Use regex to detect amount, date, GSTIN, and category hints
4. Return both raw text and parsed data

OCR output can help with

- Bill amount
- Bill date
- GSTIN
- Category guess
- Transaction creation from scanned data

Important limitation

OCR is rule-based after raw text extraction, so vendor name, exact GST values, and noisy bill layouts may still require manual review.

22. Voice Input and Live Browser Recording

Two voice paths exist

- Voice file upload path
- Live microphone recording path

Live browser recording flow

1. Browser asks for mic permission
2. `MediaRecorder` captures audio chunks in browser memory
3. Chunks are stored temporarily in `chunksRef.current`
4. On stop, chunks are combined into a Blob
5. Blob is sent to `/api/input/voice`
6. Backend writes a temporary audio file
7. Whisper transcribes the file
8. Temporary file is deleted
9. Transcript is returned to the frontend draft box

Important storage answer

Live mic recordings are not permanently stored as audio files by default.

What is stored:

- In browser: temporary chunks in memory
- In backend: temporary file in temp folder during transcription
- Permanently: only transcript text if user sends it as a message

> **Best Interview Answer for Voice Storage**
> Live browser audio is first kept in memory on the frontend, then sent to the backend for temporary transcription. The audio file is not stored permanently by default; only the resulting transcript may be saved as part of the chat.

23. Chat Persistence

The chat system stores:

- Conversations
- Messages
- User and assistant roles

Flow:

1. User sends a message
2. Backend creates a new conversation if needed
3. User message is stored
4. Assistant answer is generated
5. Assistant message is stored
6. Session history can be reopened later

This makes the chat state persistent rather than temporary.

24. Multimodal API Surface

Important endpoints in `app/api/multimodal_router.py`:

- `POST /api/upload`
- `POST /api/ocr`
- `POST /api/stt`
- `POST /api/input/text`
- `POST /api/input/voice`
- `POST /api/input/image`
- `GET /api/multimodal/health`

These endpoints allow the system to accept text, stored files, raw browser audio, and raw browser images.

25. Local LLM Usage

TinyLlama is used mainly for concise response generation and formatting.

Important design choice:

- MySQL answers come from deterministic logic
- GST answers come from retrieved document context
- TinyLlama is not used as the only answer source for everything

This is a safer design for business data because it reduces hallucination risk.

26. Commands to Run the System

First-time setup

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
npm --prefix frontend install
```

Database setup

```
python init_db.py
python scripts\migrate_db.py
```

Development mode

Backend terminal:

```
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Frontend terminal:

```
npm --prefix frontend run dev
```

Production-like local run

```
npm --prefix frontend run build
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

27. Important Development Fixes and Troubleshooting Lessons

Problem 1: Frontend and backend were running, but signup did not work correctly in dev

Root cause:

- Frontend ran on `http://localhost:5173`
- Backend ran on `http://127.0.0.1:8000`
- Axios used `window.location.origin`, so API calls were going to the Vite dev server origin instead of the backend unless proxied

Fix:

- Add a Vite dev proxy for `/api` to `http://127.0.0.1:8000`

Problem 2: Environment confusion

Observation:

- `python` and `uvicorn` were coming from the global Python installation, not from the repo `venv`

Why it matters:

- The active backend environment and the inspected local virtual environment can differ
- This can confuse debugging if packages or runtime behavior do not match

Problem 3: Signup succeeded but login returned `401 Unauthorized`

Meaning:

- Account creation worked
- Login credentials did not match exactly, or the environment mismatch caused confusion

Important conclusion:

- A `POST /api/auth/register 200 OK` means user creation succeeded
- A later `POST /api/auth/login 401 Unauthorized` means email or password verification failed

28. Common Troubleshooting Checklist

Problem	Likely cause	What to check
Signup fails	API not reaching backend	Vite proxy and backend logs
Login fails with 401	Wrong credentials or hash mismatch	Exact email, exact password, stored user row
Protected pages redirect	Missing token	Local storage token and auth provider
Voice does not work	Mic permission or Whisper issue	Browser permission and STT config
OCR weak results	Image quality or OCR limitations	Better image and manual field review
Assistant gives no DB answer	Query not strongly SQL-like	Try clearer business wording
GST answer weak	Poor retrieval match	Improve document quality or query wording

29. Best Interview Strengths to Highlight

- Full-stack integration, not just AI feature demo
- Proper separation of routes, models, schemas, and services
- Real authentication and authorization flow
- Two-database architecture with clear workload split
- Deterministic business-data answering through MySQL
- RAG-based GST knowledge retrieval
- OCR and speech-to-text integration
- Persistent chat history
- Central orchestrator deciding the right answer path

30. Honest Limitations You Can Mention Positively

- `Base.metadata.create_all()` is useful for local setup, but formal migrations are better in production
- OCR parsing is partly regex-based, so some fields may need manual correction
- Query classification is heuristic-driven, not a large ML classifier
- RAG answer generation is concise and lightweight rather than deeply generative
- Prototype folders exist because the project evolved in steps before integration

Good way to phrase this

I first built the components separately, such as query classification and multimodal processing, and then integrated them into one end-to-end system. That helped me validate each module before combining them.

31. Ready-Made Answers for Viva

What is your project?

Vyapar Sathi is an AI-powered business assistant for small businesses. It helps users manage transactions, scan bills, ask business questions, and get GST-related answers.

Which databases are used in your project?

My project uses MySQL for structured business data and ChromaDB for GST document embeddings and semantic retrieval.

Why did you use two databases?

Because the project has two different workloads: relational business operations and semantic document retrieval.

How does the assistant answer questions?

The assistant first classifies the query. If it is a business-data question, it uses MySQL through the DataService. If it is a GST knowledge question, it uses ChromaDB through the RAGService. If it is mixed, it combines both.

What happens when a user uploads a bill?

The image is uploaded, stored in the filesystem, passed through OCR, and the extracted fields can then be reviewed and saved as a transaction.

How does live voice input work?

The browser records audio with MediaRecorder, stores chunks in memory, sends them to the backend, Whisper converts them to text, and only the transcript is kept if the user submits it.

32. Cheat Sheet: Files You Must Remember

File	Why It Matters
`app/main.py`	Backend entry point
`app/core/database.py`	MySQL engine and session setup
`app/core/security.py`	Hashing, JWT, current user
`app/api/auth_router.py`	Register and login
`app/api/transaction_router.py`	Transaction CRUD and dashboard summary
`app/api/chat_router.py`	Chat sessions, history, and send
`app/api/multimodal_router.py`	Upload, OCR, STT, direct multimodal input
`app/models/schema.py`	SQLAlchemy tables

File	Why It Matters
`app/schemas/schemas.py`	Pydantic request and response schemas
`app/services/data_service.py`	MySQL business logic
`app/services/classifier_service.py`	Query type detection
`app/services/rag_service.py`	GST retrieval
`app/services/ocr_service.py`	Bill OCR
`app/services/stt_service.py`	Whisper transcription
`app/services/chat_orchestrator.py`	Central AI orchestration
`frontend/src/App.jsx`	Frontend route map

File	Why It Matters
`frontend/src/lib/api.js`	Frontend API layer
`frontend/src/providers/auth-provider.jsx`	Frontend auth state
`frontend/src/pages/assistant-page.jsx`	Assistant UI and live recording flow

33. Cheat Sheet: Important Endpoints

Endpoint	Purpose
`POST /api/auth/register`	Create account
`POST /api/auth/login`	Login and receive JWT
`POST /api/auth/logout`	Logout response
`GET /api/transaction-types`	Frontend dropdown values
`POST /api/transactions`	Add transaction
`GET /api/transactions`	List transactions
`PUT /api/transactions/{id}`	Update transaction
`DELETE /api/transactions/{id}`	Delete transaction

Endpoint	Purpose
`GET /api/dashboard/summary`	KPI summary
`POST /api/chat`	Send assistant message
`GET /api/chat/sessions`	List chat sessions
`GET /api/chat/history/{sessionId}`	Load session history
`DELETE /api/chat/{sessionId}`	Delete a chat
`POST /api/upload`	Upload file
`POST /api/ocr`	OCR on uploaded file
`POST /api/stt`	STT on uploaded file

Endpoint	Purpose
`POST /api/input/voice`	Direct live mic transcription

34. Cheat Sheet: Interview Keywords

- React SPA
- FastAPI backend
- SQLAlchemy ORM
- Pydantic validation
- JWT authentication
- MySQL relational database
- ChromaDB vector database
- RAG
- OCR
- Whisper STT
- Multimodal assistant
- Query classification
- Orchestration layer
- Persistent chat sessions

35. Final Memory Map

If you remember only one structure, remember this:

1. User logs in through JWT authentication
2. Frontend sends requests to FastAPI
3. MySQL stores business records
4. ChromaDB stores GST knowledge embeddings
5. Assistant classifies the query
6. Data questions go to MySQL
7. GST questions go to ChromaDB
8. Bill images go to OCR
9. Voice goes to Whisper STT
10. Final answer returns to the user and chat can be stored

Final Master Answer

Vyapar Sathi is a full-stack AI business assistant where React handles the user interface, FastAPI handles APIs and orchestration, MySQL stores structured business data, ChromaDB powers GST semantic retrieval, OCR reads bills, Whisper handles voice, and the assistant intelligently chooses the correct answer path.